```
# Importando Bibliotecas

import warnings
import itertools
import seaborn as sns
import matplotlib.pylab as plt
import pandas as pd
import numpy as np
import statsmodels.api as sm

from sklearn.base import BaseEstimator, TransformerMixin;



# Instalando modelos ARIMA

# !pip install h5py
# !pip install typing-extensions
# !pip install wheel
!pip install -Iv pmdarima==2.0.1


from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

```
# DATAFRAME
url = 'drive/MyDrive/SERIE_MENSAL_SOBRADINHO_1.csv'
df = pd.read_csv(url)
df.head()
```

	ANO	JAN	FEV	MAR	ABR	MAI	JUN	JUL	AGO	SET	OUT	NOV	DEZ	MED
0	1931	4676	4805	6630	7333	4467	2397	1870	1619	1448	1376	1935	2207	3397
1	1932	4036	4531	3214	1976	1425	1292	1176	995	905	982	2107	3594	2186
2	1933	5014	5516	3429	2692	1905	1357	1250	1072	915	1194	1799	3336	2457
3	1934	6110	4222	2298	1615	1785	1167	997	897	799	877	905	1605	1940
4	1935	4519	5442	5408	4716	3740	2010	1557	1237	1055	1079	1178	2585	2877

```
# Função para remoção de Colunas
class AjustarColunas(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = columns
    def ajustar(self, inputDF, y=None):
```

```
            return self
        def transform(self, inputDF):
            # Criando dataframe apenas com as colunas necessárias
            return inputDF.copy().drop(labels=self.columns, axis='columns')

    # 1. Remoção da coluna MED
    rm_columns = AjustarColunas(columns=['MED'])
    rm_columns.ajustar(inputDF=df)
    data = pd.DataFrame.from_records(data=rm_columns.transform(inputDF=df),)

    # 2. Remoção das 3 ultimas linhas
    data.drop(data.tail(3).index,inplace=True)

    # 3.Transformar a estrutura da data
    t = { 'DATE': [], 'VALUE': [] }
    months = {
    'JAN': '01',
    'FEV': '02',
    'MAR': '03',
    'ABR': '04',
    'MAI': '05',
    'JUN': '06',
    'JUL': '07',
    'AGO': '08',
    'SET': '09',
    'OUT': '10',
    'NOV': '11',
    'DEZ': '12',
    }

    for indexRow, row in data.iterrows():
        for indexCol in row.iteritems():
            if(indexCol[0] != 'ANO'):
                t['DATE'].append(months[indexCol[0]] + '-01-' + row['ANO'] )
                t['VALUE'].append(indexCol[1])

    data = pd.DataFrame(data=t)
    data['DATE'] = pd.to_datetime(data['DATE'])
    data.set_index('DATE', inplace=True)

    print(data)

    # data.plot(figsize=(15, 6))

                   VALUE
         DATE
         1931-01-01   4676
         1931-02-01   4805
         1931-03-01   6630
         1931-04-01   7333
         1931-05-01   4467
         ...           ...
```

```
2016-08-01    386
2016-09-01    313
2016-10-01    354
2016-11-01    704
2016-12-01   1989

[1032 rows x 1 columns]
```

```
df = data
```

```
from dateutil.parser import parse
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
plt.rcParams.update({'figure.figsize': (10, 7), 'figure.dpi': 120})

# Import as Dataframe
# df = pd.read_csv('drive/MyDrive/SERIE_MENSAL_SOBRADINHO_1.csv')
#df.head()
print(df)
```

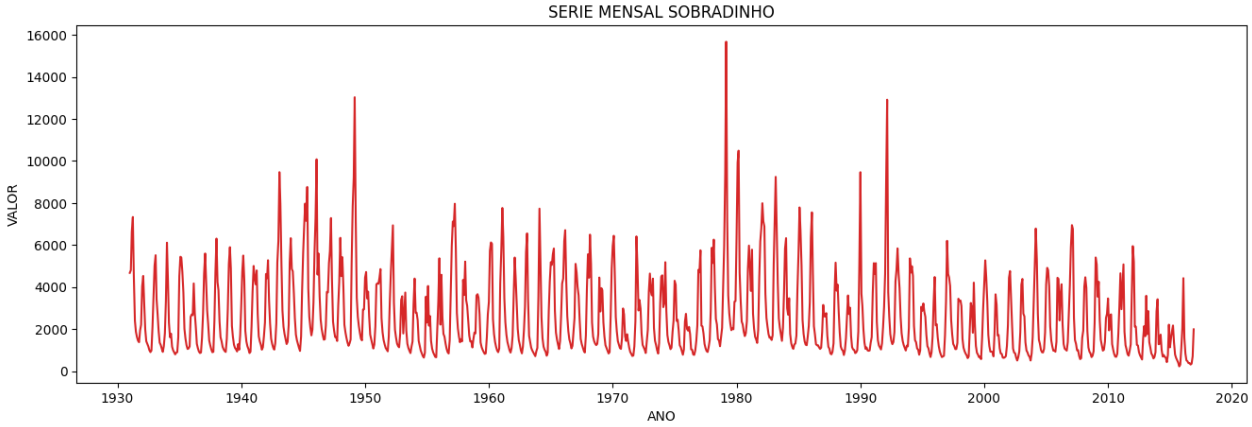
```
                VALUE
DATE
1931-01-01   4676
1931-02-01   4805
1931-03-01   6630
1931-04-01   7333
1931-05-01   4467
...           ...
2016-08-01    386
2016-09-01    313
2016-10-01    354
2016-11-01    704
2016-12-01   1989

[1032 rows x 1 columns]
```

```
import matplotlib.pyplot as plt
# df = pd.read_csv('drive/MyDrive/SERIE_MENSAL_SOBRADINHO_1.csv', parse_dates=['ANO']

#Draw plot
def plot_df(df, x, y, title="", xlabel='ANO', ylabel='VALOR', dpi=100):
    plt.figure(figsize=(16,5), dpi=dpi)
    plt.plot(x, y, color='tab:red')
    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)
    plt.show()

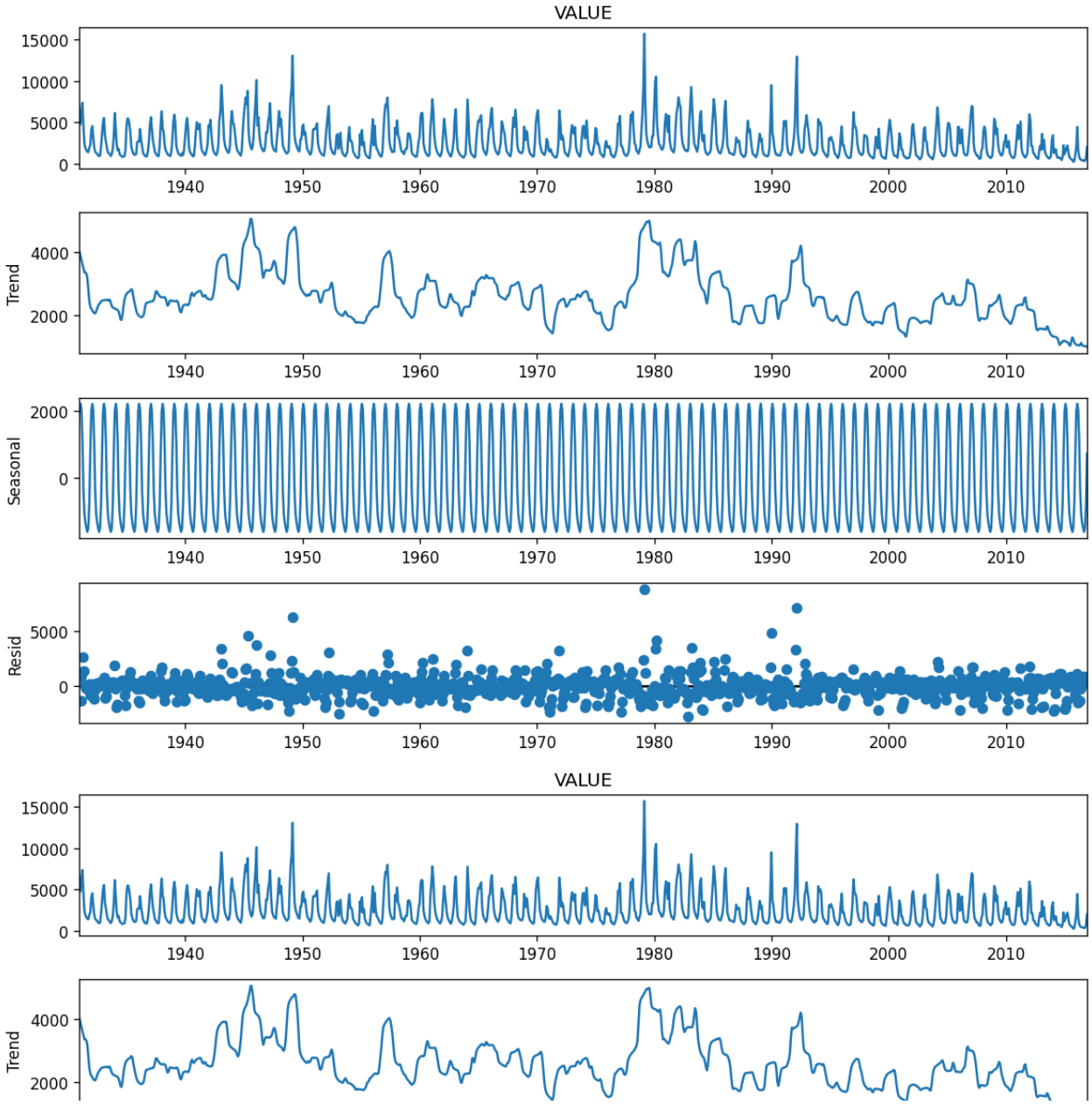
plot_df(df, x=df.index, y=df.VALUE, title='SERIE MENSAL SOBRADINHO')
```




```
from statsmodels.tsa.seasonal import seasonal_decompose
from dateutil.parser import parse

# Additive Decomposition
result_add = seasonal_decompose(df['VALUE'], model='additive', extrapolate_trend='freq

result_add.plot()
```




```
# Multiplicative Decomposition
result_mul = seasonal_decompose(df['VALUE'], model='multiplicative', extrapolate_trend

result_mul.plot()
```